Units 2 & 3: Searching, Algorithm Analysis, and Sorting

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Unit 2: Searching and Algorithm Analysis

Sections 2.1 – 2.10

You need to find a contact in your phone 🔍

Your phone has 500 contacts. They're sorted alphabetically.

How does your phone find "Zara Williams" so fast?
What strategy would you use?

2.1 Searching and Algorithms

Searching is one of the most fundamental operations in computing — find an item in a collection.

Two classic strategies:

- **Linear search** — check every item one by one from the start
- **Binary search** — repeatedly halve the search space (requires sorted data)

The choice between them is not just stylistic. At large scale it is the difference between *practical* and *impossible*.

🤔 You have a sorted list of 1,024 numbers.

Linear search checks up to 1,024 elements in the worst case.

If you always jump to the middle and discard the wrong half — how many steps do you need at most?

2.2 Binary Search

Each step cuts the remaining search space **in half**.

```
Array: [1, 3, 5, 7, 9, 11, 13, 15]    Target: 11

Step 1: mid = index 3 → value 7   → 11 > 7, search right half
Step 2: mid = index 5 → value 11  → FOUND ✓
```

For 1,024 elements: 1024 → 512 → 256 → 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1

At most 10 steps. That is $\log_2(1024) = 10$.

> Binary search is O(log n). Every time n doubles, it takes just one more step.

🤔 What does that look like in C++?

You've seen the concept. Now think about the implementation.

What variables do you need to track in binary search?
What is the loop condition?

2.3 C++: Linear and Binary Search

```cpp
// O(n) — checks every element
int linearSearch(const vector<int>& v, int target) {
    for (int i = 0; i < (int)v.size(); i++)
        if (v[i] == target) return i;
    return -1;
}

// O(log n) — halves the search space each step
int binarySearch(const vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;  // avoids integer overflow
        if      (v[mid] == target) return mid;
        else if (v[mid] < target)  lo = mid + 1;
        else                       hi = mid - 1;
    }
    return -1;
}
```

🤔 Some operations always take the same amount of time, no matter how big the input is.

`v[0]` on a million-element vector takes the same time as on a 5-element vector.

Can you think of three other operations like this? What do they have in common?

2.4 Constant Time Operations

An operation is **O(1) — constant time** if its runtime does not depend on input size `n`.

Operation	Why it's O(1)
`v[i]` — array access by index	Memory address = base + i × size. Direct.
`v.size()` — vector size	Stored as a field. No counting needed.
`s.push(x)` — stack push	Adds to one end. No traversal.
`int x = 5` — variable assignment	One memory write.
`a + b` — arithmetic	One CPU instruction.

O(1) does **not** mean instantaneous — it means the time is *bounded by a constant* regardless of n.

Two algorithms both work correctly.

Algorithm A: $5n^2 + 3n + 100$ steps

Algorithm B: $1000n$ steps

For small n (say n = 5), which is faster? For large n (say n = 10,000)?

2.5 Growth of Functions and Complexity

For small n, constants dominate. For large n, the **growth rate** dominates everything.

n	$5n^2 + 3n + 100$	1000n
5	240	5,000
100	50,400	100,000
1,000	5,003,100	1,000,000
10,000	500,030,100	10,000,000

Algorithm A is faster for small n. But at large n, Algorithm B (O(n)) **utterly crushes** Algorithm A ($O(n^2)$).

> The growth rate is the only thing that matters at scale. Constants are a rounding error.

Growth Rates Visualized (From slowest to fastest growing)

```
O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ)
```

At n = 64:

Complexity	Operations
O(1)	1
O(log n)	6
O(n)	64
O(n log n)	384
$O(n^2)$	4,096
$O(2^n)$	18,446,744,073,709,551,616

That last number is larger than all the grains of sand on Earth.

How do we formally express "this algorithm grows at most as fast as n^2"?

We want a notation that captures the **growth rate** while ignoring **constants and lower-order terms**.

What would you want such a notation to express?
What would it throw away?

2.6 O Notation

Big-O expresses an upper bound on growth rate.

Rules for simplification:

1. Drop all lower-order terms: $O(n^2 + n + 1) \rightarrow \mathbf{O(n^2)}$
2. Drop all constant multipliers: $O(3n) \rightarrow \mathbf{O(n)}$
3. Keep only the dominant term

Practice:

- $T(n) = 7n^3 + 2n^2 + 50 \rightarrow \mathbf{O(n^3)}$
- $T(n) = 4 \log n + 12 \rightarrow \mathbf{O(\log n)}$
- $T(n) = n \cdot \log n + n \rightarrow \mathbf{O(n \log n)}$

🤔 Here is a fragment of code.

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        doWork();   // O(1)
```

Without running it — what is the time complexity? How did you figure it out?

2.7 Algorithm Analysis

Counting operations from code structure:

Pattern	Complexity	Reason
Single loop 0..n	O(n)	n iterations
Nested loops 0..n × 0..n	$O(n^2)$	n × n iterations
Loop that halves: `i *= 2`	O(log n)	halves each step
Nested: outer O(n), inner O(log n)	O(n log n)	multiplied
Statement outside a loop	O(1)	independent of n

Key insight: For sequential code, *add* complexities. For nested code, *multiply* them.

Analyzing a Real Function

```
int findMax(const vector<int>& v) {  // n = v.size()
    int max = v[0];                  // O(1)
    for (int i = 1; i < n; i++)     // loop runs n-1 times → O(n)
        if (v[i] > max)              // O(1) per iteration
            max = v[i];              // O(1) per iteration
    return max;                      // O(1)
}
// Total: O(1) + O(n) + O(1) = O(n)
```

Best case: O(n) — still must check every element to be sure.

Worst case: O(n) — same loop regardless of input.

Average case: O(n) — no shortcut exists.

`findMax` is unconditionally O(n). The loop cannot exit early.

🤔 What is a recursive function?

`factorial(5)` calls `factorial(4)`, which calls `factorial(3)`...

In your own words: what makes a function recursive? What stops it from running forever?

2.8 Recursive Definitions

A **recursive definition** defines something in terms of a smaller version of itself, plus a **base case** that stops the recursion.

Factorial:

- Base case: $0! = 1$
- Recursive case: $n! = n \times (n-1)!$

Fibonacci:

- Base case: $\text{fib}(0) = 0$, $\text{fib}(1) = 1$
- Recursive case: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Recursive Definitions

Two requirements for a valid recursive definition:

1. There must be at least one **base case** (no self-reference)
2. Every recursive call must move **closer** to the base case

Without these, you get infinite recursion — and eventually, a stack overflow.

🤔 Can you write recursive binary search?

The iterative version used a `while` loop with `lo` and `hi`.

How would you express the same logic as a recursive function? What is the base case?

2.9 Recursive Algorithms

```
// Recursive factorial
int factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);   // recursive case
}

// Recursive binary search
int binarySearch(const vector<int>& v, int target, int lo, int hi) {
    if (lo > hi) return -1;                      // base case: not found
    int mid = lo + (hi - lo) / 2;
    if      (v[mid] == target) return mid;       // base case: found
    else if (v[mid] < target)
        return binarySearch(v, target, mid+1, hi);  // recurse right
    else
        return binarySearch(v, target, lo, mid-1);  // recurse left
}
```

🤔 How do we find the Big-O of a recursive function?

For a loop it's easy — count iterations.

But a recursive function calls *itself*. How would you count the total work?

Think about `factorial(n)` — how many total function calls are made?

2.10 Analyzing Recursive Time Complexity

We use **recurrence relations** — equations that express the cost of a problem in terms of the cost of smaller subproblems.

Factorial:

- $T(n) = T(n-1) + O(1)$ → one recursive call plus constant work
- Unrolling: $T(n) = T(n-1) + 1 = T(n-2) + 2 = \ldots = T(0) + n =$ **$O(n)$**

Recursive binary search:

- $T(n) = T(n/2) + O(1)$ → one recursive call on half the data
- Unrolling: $T(n) = T(n/4) + 2 = T(n/8) + 3 = \ldots = T(1) + \log_2 n =$ **$O(\log n)$**

Analyzing Recursive Time Complexity

Naïve Fibonacci:

- $T(n) = T(n-1) + T(n-2) + O(1)$ → *two* recursive calls
- This grows as **$O(2^n)$** — catastrophically slow

> The branching factor of recursion determines its complexity. One branch → $O(n)$ or $O(\log n)$. Two branches → often $O(2^n)$.

The Fibonacci Call Tree

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2) — fib(1), fib(0)
│   │   └── fib(1)
│   └── fib(2)
│       ├── fib(1)
│       └── fib(0)
└── fib(3)
    ├── fib(2)
    │   ├── fib(1)
    │   └── fib(0)
    └── fib(1)
```

`fib(5)` makes **15 total calls**. `fib(50)` makes over **2 trillion**. The same subproblems are recomputed over and over — a problem solved by dynamic programming!

Unit 3: Sorting Algorithms

Sections 3.1 – 3.15

Why is sorting such a big deal?

Searching an unsorted list of 1 million items takes up to 1 million comparisons.

Sorting it first takes some work — but then every future search takes only ~20 comparisons.

When is it worth sorting first? What are we really trading?

3.1 Sorting: Introduction

Sorting arranges a collection into a defined order (usually ascending or descending).

Why it matters:

- Enables **binary search** — O(log n) vs O(n)
- Required by many algorithms (merge, divide-and-conquer)
- Makes data human-readable and debuggable

Sorting: Introduction

Key properties to compare sorting algorithms:

Property	What it means
Time complexity	How many comparisons/swaps?
Space complexity	Does it need extra memory?
Stable	Preserves original order of equal elements?
In-place	Sorts within the original array (O(1) extra space)?
Adaptive	Faster when data is already partially sorted?

Here's an unsorted array: `[64, 25, 12, 22, 11]`

Imagine you must sort it by scanning for the smallest element and moving it to the front — then repeat.

Walk through the first two passes. What does the array look like after each?

3.2 Selection Sort

Idea: Find the minimum element in the unsorted portion and swap it into position.

```
[64, 25, 12, 22, 11]   → find min (11) → swap with index 0
[11, 25, 12, 22, 64]   → find min (12) → swap with index 1
[11, 12, 25, 22, 64]   → find min (22) → swap with index 2
[11, 12, 22, 25, 64]   → find min (25) → swap with index 3
[11, 12, 22, 25, 64]   ✓ sorted
```

- **Time:** $O(n^2)$ always — always scans the full unsorted portion
- **Space:** $O(1)$ — in-place
- **Stable:** No — swapping can move equal elements out of order
- **Strength:** Very few swaps (at most $n-1$)

3.3 C++: Selection Sort

```
void selectionSort(vector<int>& v) {
    int n = v.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)   // find minimum in v[i..n-1]
            if (v[j] < v[minIdx])
                minIdx = j;
        swap(v[i], v[minIdx]);             // place minimum at front
    }
}
```

Trace the loops:

- Outer loop runs n–1 times (positions 0 through n–2)
- Inner loop runs n–1–i times for each i
- Total comparisons: (n–1) + (n–2) + ... + 1 = n(n–1)/2 = **$O(n^2)$**

🤔 A different strategy: build a sorted section from the left.

Instead of finding the global minimum, take the next unsorted element and *insert* it into its correct position in the already-sorted section.

How is this different from selection sort? What happens if the array is already mostly sorted?

3.4 Insertion Sort

Idea: Maintain a sorted left portion. Take new element and insert into the right position.

```
[64, 25, 12, 22, 11]   start
[25, 64, 12, 22, 11]   insert 25 → shift 64 right
[12, 25, 64, 22, 11]   insert 12 → shift 25, 64 right
[12, 22, 25, 64, 11]   insert 22 → shift 25, 64 right
[11, 12, 22, 25, 64]   insert 11 → shift everything right
```

- **Time:** $O(n^2)$ worst case, **O(n) best case** (already sorted!)
- **Space:** O(1) — in-place
- **Stable:** Yes
- **Adaptive:** Yes — extremely fast on nearly-sorted data

Insertion sort is the algorithm humans use intuitively when sorting a hand of cards.

3.5 C++: Insertion Sort

```cpp
void insertionSort(vector<int>& v) {
    int n = v.size();
    for (int i = 1; i < n; i++) {
        int key = v[i];      // element to be inserted
        int j = i - 1;
        while (j >= 0 && v[j] > key) {  // shift larger elements right
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = key;      // insert at correct position
    }
}
```

C++: Insertion Sort

Why O(n) best case?

If the array is already sorted, `v[j] > key` is never true — the `while` loop never executes. The outer `for` loop runs n–1 times doing O(1) work each. Total: **O(n)**.

This makes insertion sort ideal for nearly-sorted data or small arrays — and it's actually used inside `std::sort` for subarrays below ~16 elements.

Selection and insertion sort are both $O(n^2)$. Can we do better?

What if instead of moving one element at a time, we could split the problem in half, sort each half, and combine?

Roughly how many operations would sorting two halves of n/2 elements each cost, compared to sorting n elements directly?

3.6 Quicksort

Idea: Choose a **pivot**, partition the array so all elements less than the pivot are on the left and all greater are on the right, then recursively sort each side.

```
[64, 25, 12, 22, 11]   pivot = 22

Partition: [12, 11] | 22 | [64, 25]

Recurse left:  [11, 12]
Recurse right: [25, 64]

Result: [11, 12, 22, 25, 64] ✓
```

Quicksort

- **Time:** O(n log n) average, **$O(n^2)$ worst case** (bad pivot choice)
- **Space:** O(log n) — call stack depth
- **Stable:** No
- **In-place:** Yes (with careful implementation)

🤔 When does quicksort hit its worst case — $O(n^2)$?

The pivot divides the array into two parts. Ideally those parts are roughly equal.

What would have to be true about the pivot for every partition to produce the most *unequal* split possible?

Quicksort: Pivot Choice Matters

Worst case: pivot is always the minimum or maximum element.

```
[1, 2, 3, 4, 5]  pivot = 1 (first element)
→ left: []  right: [2, 3, 4, 5]   ← one side has n-1 elements!
→ left: []  right: [3, 4, 5]
→ ...
```

Each partition does O(n) work but only shrinks the problem by 1. Result: $O(n^2)$. This happens on already-sorted data if you always pick the first element as pivot.

Common fixes:

- **Random pivot** — pick a random index
- **Median-of-three** — pivot = median of first, middle, last elements

Average-case performance with good pivot selection is reliably O(n log n).

3.7 C++: Quicksort

```
int partition(vector<int>& v, int lo, int hi) {
    int pivot = v[hi];   // choose last element as pivot
    int i = lo - 1;
    for (int j = lo; j < hi; j++) {
        if (v[j] <= pivot) {
            i++;
            swap(v[i], v[j]);
        }
    }
    swap(v[i + 1], v[hi]);
    return i + 1;         // pivot is now at its final position
}

void quickSort(vector<int>& v, int lo, int hi) {
    if (lo < hi) {
        int p = partition(v, lo, hi);
        quickSort(v, lo, p - 1);   // sort left of pivot
        quickSort(v, p + 1, hi);   // sort right of pivot
    }
}
```

🤔 Merge sort takes a different approach.

Instead of partitioning in place, it splits the array in half, sorts each half, then *merges* the two sorted halves.

What does merging two sorted arrays look like? How would you do it by hand?

3.8 Merge Sort

Idea: Split until each subarray has 1 element (trivially sorted), then merge pairs back up.

```
[64, 25, 12, 22, 11]

Split:  [64, 25, 12]        [22, 11]
Split:  [64, 25] [12]       [22] [11]
Split:  [64] [25] [12]      [22] [11]
Merge:  [25, 64]  [12]      [11, 22]
Merge:  [12, 25, 64]        [11, 22]
Merge:  [11, 12, 22, 25, 64] ✓
```

- **Time:** O(n log n) — always, best and worst case
- **Space:** O(n) — needs a temporary array for merging
- **Stable:** Yes
- **Not in-place** — the O(n) extra space is the tradeoff for guaranteed O(n log n)

3.9 C++: Merge Sort

```
void merge(vector<int>& v, int lo, int mid, int hi) {
    vector<int> tmp(hi - lo + 1);
    int i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi)          // merge two sorted halves
        tmp[k++] = (v[i] <= v[j]) ? v[i++] : v[j++];
    while (i <= mid)  tmp[k++] = v[i++]; // copy leftover left
    while (j <= hi)   tmp[k++] = v[j++]; // copy leftover right
    for (int x = 0; x < k; x++)
        v[lo + x] = tmp[x];
}

void mergeSort(vector<int>& v, int lo, int hi) {
    if (lo >= hi) return;                // base case: 1 element
    int mid = lo + (hi - lo) / 2;
    mergeSort(v, lo, mid);               // sort left half
    mergeSort(v, mid + 1, hi);           // sort right half
    merge(v, lo, mid, hi);              // merge sorted halves
}
```

🤔 Quicksort and merge sort are both O(n log n). When would you prefer one over the other?

Quicksort is in-place but has an $O(n^2)$ worst case.

Merge sort uses O(n) extra memory but is always O(n log n).

For each scenario, which would you choose and why?

1. Sorting records in a database where stability matters
2. Sorting a large array on a system with very limited RAM

Quicksort vs. Merge Sort

Property	Quicksort	Merge Sort
Average time	O(n log n)	O(n log n)
Worst-case time	$O(n^2)$	O(n log n)
Space	O(log n) stack	O(n) extra array
Stable	No	Yes
Cache-friendly	Yes — sequential access	Less so
Used in practice	`std::sort` (hybrid)	External sorting, linked lists

Quicksort vs. Merge Sort

Rule of thumb:

- Random data in memory → quicksort (faster in practice due to cache locality)
- Guaranteed worst case needed → merge sort
- Sorting linked lists → merge sort (no random access needed)
- Stability required → merge sort

🤔 What if insertion sort and merge sort had a child?

Insertion sort is fast on small arrays and nearly-sorted data.

Merge sort is fast on large arrays.

Could you design a sorting algorithm that uses one strategy for small subarrays and another for large ones?

3.10 Shell Sort

Idea: Generalize insertion sort by sorting elements that are *h* positions apart first. Gradually shrink h to 1 (a final insertion sort pass on a nearly-sorted array).

```
Array:    [64, 25, 12, 22, 11, 90, 37, 8]
Gap h=4:  sort [64,11], [25,90], [12,37], [22,8]
          → [11, 25, 12, 8, 64, 90, 37, 22]
Gap h=2:  sort every-other element
          → [11, 8, 12, 22, 37, 25, 64, 90]  (approx)
Gap h=1:  final insertion sort (nearly sorted → fast!)
          → [8, 11, 12, 22, 25, 37, 64, 90] ✓
```

- **Time:** $O(n \log^2 n)$ with good gap sequences (better than $O(n^2)$)
- **Space:** $O(1)$ — in-place
- **Stable:** No
- Complexity depends on the gap sequence chosen

3.11 C++: Shell Sort

```cpp
void shellSort(vector<int>& v) {
    int n = v.size();
    // Knuth's gap sequence: 1, 4, 13, 40, 121, ...
    int h = 1;
    while (h < n / 3) h = 3 * h + 1;

    while (h >= 1) {
        // h-sort the array (insertion sort with gap h)
        for (int i = h; i < n; i++) {
            int key = v[i];
            int j = i - h;
            while (j >= 0 && v[j] > key) {
                v[j + h] = v[j];
                j -= h;
            }
            v[j + h] = key;
        }
        h /= 3;   // shrink gap
    }
}
```

🤔 All the sorts we've seen compare elements to decide their order.

What if the data isn't compared at all — what if we use the *value* of each element directly to place it?

Can you think of how you'd sort a deck of cards by suit and rank without comparing two cards to each other?

3.12 Radix Sort

Idea: Sort digit-by-digit (or character-by-character), from least significant to most significant. Uses counting sort as a subroutine.

```
Input:  [170, 45, 75, 90, 802, 24, 2, 66]
Sort by 1s digit:  [170, 90, 802, 2, 24, 45, 75, 66]
Sort by 10s digit: [802, 2, 24, 45, 66, 170, 75, 90]
Sort by 100s digit:[2, 24, 45, 66, 75, 90, 170, 802] ✓
```

- **Time:** O(nk) where k = number of digits — effectively O(n) for fixed-size integers
- **Space:** O(n + k)
- **Stable:** Yes (when using stable counting sort)
- **Not comparison-based** — not bounded by O(n log n) lower bound

Radix sort is the algorithm behind postal sorting machines and old IBM card sorters.

3.13 C++: Radix Sort

```
void countingSort(vector<int>& v, int exp) {
    int n = v.size();
    vector<int> output(n), count(10, 0);

    for (int i = 0; i < n; i++)          // count occurrences
        count[(v[i] / exp) % 10]++;
    for (int i = 1; i < 10; i++)         // cumulative count
        count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) { // build output (stable)
        output[count[(v[i] / exp) % 10] - 1] = v[i];
        count[(v[i] / exp) % 10]--;
    }
    v = output;
}

void radixSort(vector<int>& v) {
    int maxVal = *max_element(v.begin(), v.end());
    for (int exp = 1; maxVal / exp > 0; exp *= 10)
        countingSort(v, exp);  // sort by current digit
}
```

🤔 We've seen six sorting algorithms. How do they compare?

Selection, insertion, quicksort, merge sort, shell sort, radix sort.

Before the next slide — which one would you use for each scenario?

1. A nearly-sorted array of 50 elements
2. 10 million integers where worst-case time must be guaranteed
3. Sorting strings alphabetically where equal strings must preserve original order

3.14 Overview of Fast Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion sort	**$O(n)$**	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Shell sort	$O(n \log n)$	$O(n \log^2 n)$	$O(n^2)$	$O(1)$	No
Quicksort	$O(n \log n)$	**$O(n \log n)$**	$O(n^2)$	$O(\log n)$	No
Merge sort	$O(n \log n)$	$O(n \log n)$	**$O(n \log n)$**	$O(n)$	Yes
Radix sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes

3.14 Overview of Fast Sorting Algorithms

Answers to the previous question:

1. Nearly-sorted, 50 elements → **insertion sort** (O(n) best case, tiny n)
2. 10M integers, guaranteed worst case → **merge sort** (O(n log n) always)
3. Stable string sort → **merge sort** (stable and O(n log n))

🤔 In C++, how do you sort with a custom comparison rule?

The default `std::sort` sorts integers in ascending order. But what if you want descending order? Or to sort structs by a specific field?

What mechanism in C++ lets you pass a custom "is less than" rule to a sorting function?

3.15 C++: Sorting with Different Operators

`std::sort` accepts a custom comparator — a function or lambda that defines the ordering.

```
#include <algorithm>
#include <vector>
#include <string>

vector<int> v = {5, 2, 8, 1, 9};

// Ascending (default)
sort(v.begin(), v.end());

// Descending — lambda returns true if a should come before b
sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

Comparator Rules

Rule: A comparator must implement **strict weak ordering**:

- Irreflexive: `comp(a, a)` is always false
- Asymmetric: if `comp(a, b)` then `!comp(b, a)`
- Transitive: if `comp(a, b)` and `comp(b, c)` then `comp(a, c)`

Violating these causes undefined behavior in `std::sort`.

LAB!

https://github.com/abuach/242students/blob/main/labs/lab2.md